Document Assistant — How It Works

A plain-language walkthrough of what this app does and how it's built, written so you can explain it to someone else (or answer questions about it) without re-reading the code.

1. Elevator pitch

Documents are ingested offline via a console command (`php artisan document:ingest`), not an upload button. The app reads a document, breaks it into small chunks, and turns each chunk into a vector (a list of numbers that captures its meaning) — all of this runs **locally, for free**, via Ollama. If the document is a PDF with charts/figures, those pages are additionally rendered to images and read by a local vision model, so chart data that plain text extraction can't see becomes answerable too.

Chat is **document-scoped**: every conversation belongs to exactly one document. When you ask a question, it's turned into a vector the same way, the most similar chunks *from that one document* are retrieved, and an LLM (Google Gemini) is asked to answer using only that retrieved context — not its general knowledge, and not other documents' content. That's what "RAG" means: **R**etrieval-**A**ugmented **G**eneration. If nothing relevant was retrieved, it's instructed to say so instead of making something up.

2. Tech stack

Layer	Technology
Backend	Laravel 13 (PHP 8.3)
Frontend	React 19 + Tailwind CSS v4, bundled by Vite
Database	SQLite (`database/database.sqlite`) — stores documents, chunk text **and** embedding vectors, conversations, messages
Vector search	None external — cosine similarity computed in PHP over chunks already in SQLite, scoped to one document at a time
Embeddings	**Ollama**, local — `nomic-embed-text`
Chart/figure vision (PDF ingestion only)	**Ollama**, local — `qwen2.5vl:3b` , fed PNG renders produced by a Python/PyMuPDF subprocess
Chat generation	Google Gemini (`gemini-3.5-flash-lite` , configurable)

There's no authentication — it's a single shared space, one instance for whoever's running it. There's no external vector database (Pinecone or otherwise): embeddings are small enough to sit as a JSON column right next to their chunk's text, and a linear cosine-similarity scan over a few hundred chunks is sub-millisecond, so a dedicated index would be solving a problem that doesn't exist at this scale.

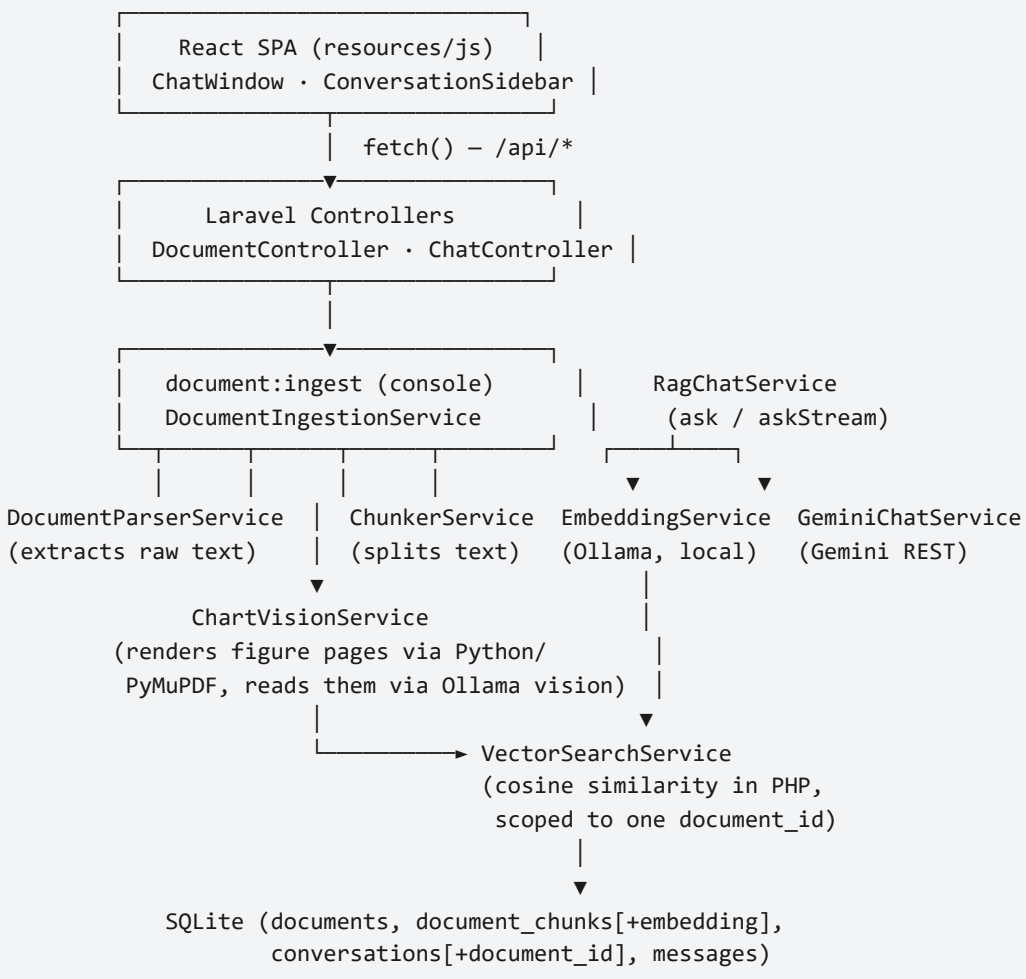
3. Key concept: what's an embedding, and why does document-scoping matter?

What an embedding is. `nomic-embed-text` doesn't generate text — its only job is: take a piece of text in, output a list of numbers (a vector) that represents its *meaning*. Text with similar meaning lands at nearby points in that vector space; unrelated text ends up far apart. "Find the most relevant chunk" becomes "find the nearest point" — that's the entire mechanism behind `VectorSearchService::search()`.

Why local (Ollama) instead of a hosted embedding API. The original version of this app called a hosted embedding API per chunk, which meant pooling/pacing requests and running into daily quota caps on large documents. A 200-page report with heavy chart content can mean hundreds of embedding calls plus dozens of multi-minute vision calls — all free and quota-free when run against a local model instead. The trade-off: ingestion speed is now bound by your own CPU (there's no managed scaling), and production deployments must run Ollama themselves too (see §10).

Why chat is document-scoped. Every `Conversation` row has a `document_id`. `VectorSearchService::search()` filters to that document's chunks only, so a question asked in one document's chat can never accidentally retrieve (or leak) another document's content — distinct from the single-shared-corpus design this app started with, where every document's chunks were searched together.

4. Architecture diagram



5. Flow #1 — Ingesting a document

Entry point: `php artisan document:ingest <path> [name] [--force]` → `IngestDocument` command → `DocumentIngestionService::ingestFromPath()`.

1. **Create the row.** A `Document` row is created with `status = processing`. The file extension is taken from the real file path (not the display name, which may have none).
2. **Parse.** `DocumentParserService` extracts the full plain text — `smalot/pdfparser` for PDF, `phpoffice/phpword` for DOCX, plain `file_get_contents` for TXT/MD.
3. **Chart/figure vision (PDF only).** `ChartVisionService::describeCharts()`: - Scans every page's extracted text for the literal word "FIGURE" — a caption heuristic, not a content/image detector. - For each matching page, shells out to `scripts/render_pdf_page.py` (PyMuPDF) to rasterize that one page to a PNG — nothing in PHP on Windows can rasterize a PDF page. - Sends the PNG to the local vision model (`qwen2.5vl:3b` via Ollama's `/api/generate`) with a prompt asking for a terse list of the chart's data points, or a `NO_CHART_DATA` sentinel if there's no numeric chart on the page. - Each page's render step and describe step are **independently wrapped in try/catch** — a single slow or failed page is logged as a warning and skipped, it never aborts the whole run (a page taking several minutes on a CPU-only machine is normal for a dense multi-panel figure). - An empty model response is retried once (sampling variance at low temperature), since that's different from the model explicitly saying "no chart here."
4. **Combine.** Any chart descriptions found are appended to the *end* of the document's full text, each tagged `[Chart/figure data from page N]`. They are not interleaved back onto their original page, so a page's prose and its chart data end up in different, separately-retrievable chunks later.
5. **Chunk.** `ChunkerService` splits the combined text into ~800-character chunks, preferring paragraph boundaries (falling back to sentence boundaries for an oversized paragraph), with an 80-character overlap between consecutive chunks so context isn't lost at a boundary.
6. **Embed.** `EmbeddingService::embedMany()` sends all chunk texts to Ollama's `/api/embed` in batches of 50 — Ollama natively batches multiple inputs per request, so unlike a hosted API there's no need for pooling, pacing, or quota handling.
7. **Store.** Each chunk's text and its embedding vector (as JSON) are bulk-inserted together into `document_chunks` in one query — there's no separate vector database to write to.
8. The `Document` row becomes `status = ready` with its final `chunk_count`. If anything in steps 2-7 throws, it's marked `status = failed` with the error message instead of losing all prior progress silently.

A document with no "FIGURE"-marked pages at all (e.g. a product spec sheet whose charts have their labels as real text, not drawn shapes) skips step 3 entirely and finishes in seconds regardless of page count — ingestion time is driven almost entirely by how many figure pages a PDF has, not its page count.

6. Flow #2 — Asking a question

Entry point: `ChatWindow.jsx` → `POST /api/chat/stream` → `ChatController::askStream()` → `RagChatService::askStream()`.

1. **Resolve the conversation.** A `Conversation` is found or created by `session_uuid`; a brand-new conversation is created with the `document_id` the frontend sent (required to start a new chat), and that document never changes for the lifetime of that conversation.
2. **Embed the question.** The user's message is turned into a vector the same way document chunks are (`EmbeddingService::embed()`).
3. **Retrieve, scoped.** `VectorSearchService::search()` loads every chunk belonging to `conversation->document_id` (and only that document), ranks them by cosine similarity against the question's vector, and takes the top 4.
4. **Build the prompt.** The retrieved chunks become the `CONTEXT` block in a system prompt (`RagChatService::SYSTEM_TEMPLATE`) naming the specific document by filename, instructing Gemini to: - answer only from that context or from facts already established earlier in the conversation, - respond with an exact "I don't have that

information in *{document}*." line otherwise, - state directly-related figures found together in the context explicitly (e.g. a baseline, a reduction, and the resulting total) rather than only answering the literal question, - keep answers to 2-4 sentences unless more detail is explicitly requested. Prior messages in the conversation are included as history, so follow-up questions work.

5. **Generate, streamed.** `GeminiChatService::generateStream()` calls Gemini's `streamGenerateContent` (Server-Sent Events) endpoint. `ChatController` re-emits those as SSE events (`meta` → `delta` * → `done`) to the browser. `ChatWindow.jsx` appends each text fragment to the message bubble live, so the answer appears progressively instead of after a multi-second wait.
6. **Persist.** Once the full answer is assembled, both the user's message and the assistant's answer (plus which sources were used) are saved as `Message` rows. `Message` "touches" its parent `Conversation` , bumping `updated_at` so the sidebar's recency grouping stays accurate.

A 429 response from Gemini containing `RESOURCE_EXHAUSTED` is distinguished from a transient failure and surfaced as `GeminiQuotaExceededException` , giving the user a specific "try again in about N hour(s)" message instead of a generic error.

There's also a non-streaming twin of this (`POST /api/chat` → `ChatController::ask()`) that returns the full answer as one JSON response — useful for API consumers or quick `curl` testing, but the UI always uses the streaming version.

7. Seeding the database

Running `document:ingest` for a large PDF with lots of charts is a multi-hour, CPU-bound job — nobody should have to repeat that just to get a working copy of the app. So the result of ingestion is also committed to the repo as seed data:

- `php artisan document:export-seed` dumps every row of `documents` and `document_chunks` (text **and** embedding vectors) to `database/seeders/data/documents.json` / `document_chunks.json` .
- `database/seeders/DocumentSeeder.php` reads those files back and bulk-inserts them, wired into `DatabaseSeeder` so a plain `php artisan db:seed` picks it up.
- `conversations` and `messages` are deliberately **not** part of this — they're live, per-environment chat data, not ingestion output, and a freshly-seeded database always starts with empty chat history.
- `database/database.sqlite` itself stays gitignored (`database/.gitignore`): what's committed is the JSON export, not the live database file. This keeps every future chat message from becoming a binary diff in git history — only a deliberate `document:export-seed` run changes the seed data.

Anyone who ingests a new document and wants to share it re-runs `document:export-seed` (a full re-export of everything currently in the tables, not an append) and commits the regenerated JSON files.

8. Switching documents mid-session

Because a `Conversation` is permanently tied to one `document_id` , there's no "change this conversation's document" operation. The frontend's document switcher (the dropdown always visible at the top of `ChatWindow.jsx`) instead starts a **new** conversation scoped to the newly picked document — the same thing clicking "New chat" and then picking a document from the empty-state picker would do. The browser's remembered `session_uuid` is cleared and replaced once the first message is sent in that new conversation.

9. File-by-file reference

Backend (`app/`)

File	Responsibility
`Console/Commands/IngestDocument.php`	`document:ingest` — the only way documents get into the system
`Console/Commands/ExportDocumentSeed.php`	`document:export-seed` — dumps `documents` / `document_chunks` to the committed seed JSON
`Http/Controllers/DocumentController.php`	List ready documents (for the picker/switcher); single "most recent" status check
`Http/Controllers/ChatController.php`	Ask a question (JSON or SSE stream), fetch history, list conversations
`Services/DocumentIngestionService.php`	Orchestrates parse → chart vision → chunk → embed → store, and deletion
`Services/DocumentParserService.php`	Extracts raw text from PDF/DOCX/TXT/MD
`Services/ChartVisionService.php`	Finds figure pages, renders them (via `scripts/render_pdf_page.py`), reads them with a local vision model
`Services/ChunkerService.php`	Splits text into overlapping chunks
`Services/EmbeddingService.php`	Calls Ollama's local embedding API (batched)
`Services/VectorSearchService.php`	Cosine-similarity search over one document's chunks, in PHP
`Services/GeminiChatService.php`	Calls Gemini's chat API (single response + SSE stream)
`Services/RagChatService.php`	The actual RAG logic: retrieve context, build the prompt, call the LLM, persist
`Exceptions/GeminiQuotaExceededException.php`	Distinguishes a Gemini quota 429 from a transient failure
`Models/Document.php`, `DocumentChunk.php`, `Conversation.php`, `Message.php`	Eloquent models

Seed data (`database/seeders/`)

File	Responsibility
`DocumentSeeder.php`	Bulk-inserts `documents` / `document_chunks` from the JSON below; wired into `DatabaseSeeder`
`data/documents.json`, `data/document_chunks.json`	Committed export of every ingested document + chunk (text and embedding vectors) — regenerated by `document:export-seed`

Supporting scripts

File	Responsibility
`scripts/render_pdf_page.py`	Renders one PDF page to a PNG via PyMuPDF — called as a subprocess since nothing in PHP on Windows can rasterize a PDF page

Frontend (`resources/js/components/`)

File	Responsibility
`App.jsx`	Top-level layout; holds session UUID, document list, and selected-document state; sets the browser tab title
`ChatWindow.jsx`	Document picker (new chat) + always-visible document switcher, message list, the SSE stream consumer, the composer
`ConversationSidebar.jsx`	Past conversations grouped by recency (Today / Previous 7 days / Older), each labeled with its document
`SourcesPanel.jsx`	Collapsible "which chunks was this answer based on" panel under each answer
`Icons.jsx`	Shared inline SVG icon set

10. Database schema (SQLite)

- **documents** — `filename` , `original_extension` , `status` (`processing` / `ready` / `failed`), `chunk_count` , `error_message`
- **document_chunks** — `document_id` (FK, cascade delete), `chunk_index` , `content` (the chunk's text), `embedding` (JSON array — the vector itself, no external store)
- **conversations** — `session_uuid` (ties a browser session to its history), `document_id` (FK, nullable, set-null on delete — which document this entire conversation is scoped to)
- **messages** — `conversation_id` (FK, cascade delete), `role` (`user` / `assistant`), `content` , `sources` (JSON, assistant messages only)

11. Common questions

"Where does the AI's answer actually come from?" Only from the 4 most relevant chunks retrieved from *that conversation's one document*, plus the recent conversation history. It's explicitly told not to use outside knowledge or other documents.

"What if I ask something not in the document?" It's instructed to reply exactly *"I don't have that information in {document name}."* rather than guess.

"Does it remember earlier messages in the conversation?" Yes — full message history for that `session_uuid` is sent with every request, so follow-ups work. History is *not* summarized or trimmed, so a very long conversation will slowly get slower/costlier (there's no cap on this today).

"Why does the answer appear word-by-word instead of all at once?" It's streamed from Gemini via Server-Sent Events, so the browser renders each fragment as it arrives instead of waiting for the whole answer.

"Can it read charts and figures in a PDF, or only text?" Both. Plain text is extracted normally; pages whose text contains a "FIGURE" caption are additionally rendered to an image and read by a local vision model (`qwen2.5vl:3b`), and that transcription is folded into the document's retrievable text. A page's prose and its chart data end up as separate chunks, not merged into one.

"Where is my data stored? Is anything sent to a third party?" Document text, chunk embeddings, and chat history all live in a local SQLite file — nothing leaves the machine for parsing, embedding, or chart reading, since those all run through local Ollama

models. The only thing sent to a third party is the retrieved context + the user's question + conversation history, sent to Google Gemini purely to generate the visible chat answer.

"Can I run this without Gemini?" Not for the chat-answer step — `GEMINI_API_KEY` is a hard dependency for `GeminiChatService`. Everything else (parsing, chunking, chart reading, embedding, retrieval) runs fully locally via Ollama and needs no external account.

"What do I need installed locally that isn't a PHP/Node dependency?" Ollama (with `nomic-embed-text` and `qwen2.5vl:3b` pulled), and Python 3 with PyMuPDF installed — the latter only matters if you're ingesting a PDF that actually contains figures; a document with none never invokes the chart-vision/Python path at all.

"What does production need that my dev machine has?" Just Ollama with `nomic-embed-text` pulled, since every chat question is embedded at query time. Getting the ingested documents there is just `php artisan migrate && php artisan db:seed` — the seed data is committed to the repo (see §7), so there's nothing to copy by hand. The vision model and Python/PyMuPDF are ingestion-only and never needed in production at all, since seeding only ever inserts already-computed text and embeddings.

"Why is `database.sqlite` gitignored if the documents are committed?" They're committed separately, on purpose. `database.sqlite` is the *live* database — it also accumulates every conversation and message as people use the app, and committing that file directly would turn every chat exchange into a binary diff in git history. The seed JSON (`database/seeders/data/`) only contains `documents` and `document_chunks` — the expensive-to-regenerate ingestion output — and is only updated by a deliberate `document:export-seed` run.

"Is there authentication?" No — anyone who can reach the app can see and query every ingested document (though each conversation only ever searches the one document it's scoped to). There's no per-user data isolation.